

Explain MLE Assignment

10/4/2026

Uploaded a file Uploaded a file Explain the assignment from scratch...

Worked for 10s Absolutely. The key is to realize that Assignment 5 is basically taking...
help me prepare for VIVA of this experiment

Absolutely. For this viva, don't try to memorize the entire notebook. You need to be able...
even more depth questions, testing the fundamentals of this experiment

Yes — let's go much deeper. For a PRML viva, the examiner can easily move beyond...



Uploaded a file

Uploaded a file

Explain the assignment from scratch <https://github.com/Karthikeyan770/AI5707-PRML-Lab/tree/main/Lab%204> this is the previous experiment code



Worked for 10s

Absolutely. The key is to realize that **Assignment 5 is basically taking what you did in Lab 4 and turning the known Gaussian parameters into an estimated/classification problem.**

Your Assignment 5 specifically asks you to do **maximum-likelihood parameter estimation**, first on generated Gaussian data and then on a real 2-D dataset. AI5707_Assignment_5

I also checked your Lab 4 repository. Your previous notebook already has the important building blocks: Gaussian data generation, covariance computation, eigendecomposition, Mahalanobis-distance contours, Gaussian discriminants, and decision-boundary plotting.

1. First understand what Assignment 5 is actually asking

The assignment has **two big parts**.

Part 1 — Synthetic data

You generate data for **three Gaussian classes**:

$$C_1 \sim N(\mu_1, \Sigma_1)$$

$$C_2 \sim N(\mu_2, \Sigma_2)$$

$$C_3 \sim N(\mu_3, \Sigma_3)$$

Then:

1. Generate about **10,000 points for each class**.
2. Try different covariance structures:
 - isotropic: $\sigma^2 I$
 - diagonal covariance
 - full covariance
3. Study both:
 - same covariance across classes
 - different covariance across classes
4. Split data:
 - **80% training**

- **20% testing**

5. Estimate μ and Σ from **training data only**.
6. Use those estimated parameters to classify the test data.
7. Plot the class boundaries and test points.
8. Produce a confusion matrix.
9. Write observations.

AI5707_Assignment_5

Part 2 — Real data

You take the provided 2-D dataset:

x, y , class label

Then:

80% $\rightarrow$ training

20% $\rightarrow$ testing

Using only the training samples, estimate the Gaussian parameters with MLE, classify the test samples, generate the confusion matrix, plot the boundaries, and record observations.

AI5707_Assignment_5

2. What is different from Lab 4?

This is the most important part.

Your **Lab 4** was primarily:

"Here is a Gaussian model. Understand its covariance, eigenvalues, eigenvectors, density contours and decision boundaries."

Your **Assignment 5** is:

*"Here is data. Can you **learn the Gaussian model from the data itself?**"*

So the flow changes from:

Lab 4

μ, Σ known

↓

Generate data

↓

Analyze distribution

↓

Plot boundary

to:

Assignment 5

Data given

↓

Estimate

$\hat{\mu}, \hat{\Sigma}$

↓

Build Gaussian classifier

↓

Classify unseen data

↓

Evaluate using confusion matrix

That is the conceptual jump.

3. Maximum Likelihood Estimation — from absolute zero

Suppose a class has samples

$$x_1, x_2, \dots, x_N$$

and we assume:

$$x_i \sim N(\mu, \Sigma)$$

But we don't know μ or Σ .

We want to find the values of μ and Σ that make the observed data **most likely**.

That's Maximum Likelihood Estimation.

4. MLE of the mean

For a set of N vectors,

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N x_i$$

In 2-D, suppose:

$$x_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

Then:

$$\hat{\mu} = \begin{bmatrix} \frac{1}{N} \sum x_i \\ \frac{1}{N} \sum y_i \end{bmatrix}$$

So in Python:

```
mu_hat = np.mean(X, axis=0)
```

For example, if your samples are:

```
(1,2)
(2,3)
(3,4)
```

then:

$$\hat{\mu} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Very simple.

5. MLE of covariance

This is the second major formula.

$$\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

Notice something important:

MLE uses N

not

$$N - 1$$

Your Lab 4 code used:

```
np.cov(X, rowvar=False, ddof=1)
```

That is the **sample covariance estimator**, which uses $N - 1$.

For Assignment 5, since the assignment explicitly asks for **maximum likelihood estimation**, we should implement the covariance with:

$$\frac{1}{N}$$

This distinction is worth remembering because your instructor could absolutely ask about it.

6. Why do we estimate parameters separately for each class?

Suppose we have three classes:

Class 1

Class 2

Class 3

We shouldn't calculate one mean for all the data.

Instead:

$$\hat{\mu}_1, \hat{\Sigma}_1$$

for Class 1,

$$\hat{\mu}_2, \hat{\Sigma}_2$$

for Class 2,

and

$$\hat{\mu}_3, \hat{\Sigma}_3$$

for Class 3.

So the classifier becomes:

$$C_k \sim N(\hat{\mu}_k, \hat{\Sigma}_k)$$

This is essentially a **Gaussian classifier / QDA-LDA framework** depending on whether the covariance matrices are shared.

7. Why does the assignment give different covariance types?

This is directly connected to your Lab 4.

The assignment asks you to study:

Case A: Isotropic

$$\Sigma = \sigma^2 I$$

Example:

$$\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix}$$

This means equal variance in both directions and zero covariance.

The distribution looks like a **circle**.

Case B: Diagonal

$$\Sigma = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix}$$

Example:

$$\Sigma = \begin{bmatrix} 9 & 0 \\ 0 & 1 \end{bmatrix}$$

Now the spread differs in the two coordinate directions.

So the distribution looks like an **axis-aligned ellipse**.

Case C: Full covariance

$$\Sigma = \begin{bmatrix} \sigma_{11}^2 & \sigma_{12} \\ \sigma_{21} & \sigma_{22}^2 \end{bmatrix}$$

For example:

$$\Sigma = \begin{bmatrix} 4 & 1.5 \\ 1.5 & 2 \end{bmatrix}$$

Now the off-diagonal terms are nonzero.

That means x and y are correlated, and the ellipse can be **rotated**.

You already saw this in Lab 4: the eigenvectors determine the orientation of the principal axes.

8. The 80/20 split

This is extremely important.

Suppose Class 1 contains 10,000 points.

You split:

8000 → training

2000 → testing

And similarly for Classes 2 and 3.

The parameter estimation happens **only on the training set**.

For example:

$$\hat{\mu}_1 = \frac{1}{8000} \sum_{i=1}^{8000} x_i$$

and

$$\hat{\Sigma}_1 = \frac{1}{8000} \sum_{i=1}^{8000} (x_i - \hat{\mu}_1)(x_i - \hat{\mu}_1)^T$$

Then the test data is kept aside.

Why?

Because otherwise you're essentially allowing the model to see the answers beforehand.

9. How classification works

Now suppose a test point is:

$$x = \begin{bmatrix} 2.1 \\ 3.5 \end{bmatrix}$$

We calculate how likely this point is under each class.

For class k :

$$p(x|C_k) = \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) \right]$$

Since we're doing 2-D:

$$d = 2$$

The classifier says:

$$\text{Predicted class} = \arg \max_k p(x|C_k)$$

assuming equal class priors.

10. Why use log probability?

The Gaussian probability can become extremely small.

Instead of directly comparing:

$$p(x|C_1), p(x|C_2), p(x|C_3)$$

we usually compare:

$$\ln p(x|C_k)$$

The discriminant can be written as:

$$g_k(x) = -\frac{1}{2} \ln |\Sigma_k| - \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)$$

There are additional constants and possibly prior terms, but with equal priors they don't affect which class wins.

Then:

```
predicted_class = np.argmax(scores)
```

11. Now the REALLY important connection: boundary shape

This assignment specifically asks you to generate data with:

Same covariance matrices

$$\Sigma_1 = \Sigma_2 = \Sigma_3$$

In that case, the quadratic terms cancel when comparing classes.

So you get:

linear decision boundaries

This is essentially **LDA-like behaviour**.

Different covariance matrices

$$\Sigma_1 \neq \Sigma_2 \neq \Sigma_3$$

Now the quadratic terms don't cancel.

So:

quadratic/curved boundaries

This is **QDA-like behaviour**.

And this directly builds on the decision-boundary experiment you already did in Lab 4.

12. What your Lab 4 code can be reused for

Your previous notebook already gives us a great starting point.

You have functions conceptually equivalent to:

```
generate_gaussian(...)
```

which creates Gaussian samples.

You already have covariance/eigenvalue analysis.

You already have Mahalanobis-distance calculations.

You already have Gaussian discriminant calculations.

You already have decision-boundary plotting.

So we **do not need to start from scratch in code**.

We need to add the missing Assignment 5 layer:

```
Lab 4
↓
Generate Gaussian data
↓
Assignment 5 addition
↓
Split into train/test
↓
Estimate  $\mu$  using MLE
↓
Estimate  $\Sigma$  using MLE
↓
Build Gaussian models
↓
Classify test data
↓
Confusion matrix
↓
Decision boundary
↓
Observations
```

Your repository's Lab 4 notebook already explicitly uses Gaussian discriminants and shows that equal covariance produces a linear boundary while unequal covariance generally produces a quadratic one.

13. Part 1(a): Generate 10,000 points for EACH class

This wording matters.

The assignment doesn't mean:

generate 10,000 total points.

It says about **10,000 points for each of three classes**.

AI5707_Assignment_5

So you'd have approximately:

$$10000 + 10000 + 10000 = 30000$$

total samples.

For example:

```
N = 10000
X1 = generate_gaussian(mu1, Sigma1, N)
X2 = generate_gaussian(mu2, Sigma2, N)
X3 = generate_gaussian(mu3, Sigma3, N)
```

Then:

```
x = np.vstack([X1, X2, X3])
y = np.concatenate([
    np.zeros(N),
    np.ones(N),
    np.full(N, 2)
])
```

So:

```
X → coordinates
y → class labels
```

14. Part 1(b): Covariance experiments

You need to repeat the experiment for the three covariance types.

A clean way is to create three sets of covariance matrices.

Experiment 1 — isotropic

```
sigma1 = sigma2 * np.eye(2)
Sigma2 = sigma2 * np.eye(2)
Sigma3 = sigma2 * np.eye(2)
```

Experiment 2 — diagonal

For example:

```
sigma1 = np.array([
    [4, 0],
    [0, 1]
])
```

and similarly for other classes.

Experiment 3 — full

```
sigma1 = np.array([
    [4, 1.2],
    [1.2, 2]
])
```

The exact parameters should follow whatever your instructor expects from the preceding experiment / assignment setup.

15. Part 1(c): Same covariance vs different covariance

This is another separate experiment.

Experiment A

```
Class 1 →  $\Sigma$   
Class 2 →  $\Sigma$   
Class 3 →  $\Sigma$ 
```

Same covariance.

Experiment B

```
Class 1 →  $\Sigma_1$   
Class 2 →  $\Sigma_2$   
Class 3 →  $\Sigma_3$ 
```

Different covariance.

This is where you'll observe the decision boundaries changing from approximately straight lines to curved boundaries.

AI5707_Assignment_5

16. Part 1(d): Estimate parameters

Suppose training samples for Class 1 are:

```
X1_train
```

Then:

```
mu1_hat = np.mean(X1_train, axis=0)
```

For covariance, explicitly implementing MLE:

```
diff = X1_train - mu1_hat  
  
Sigma1_hat = (  
    diff.T @ diff  
    ) / len(X1_train)
```

Same for Class 2 and Class 3.

So after this step, the model contains:

```
Class 1:  
     $\mu_1$   
     $\Sigma_1$   
  
Class 2:  
     $\mu_2$   
     $\Sigma_2$   
  
Class 3:  
     $\mu_3$   
     $\Sigma_3$ 
```

This is the heart of the assignment.

17. Part 1(e): Classify the test set

For every test point:

$$x$$

calculate:

$$g_1(x), g_2(x), g_3(x)$$

and pick the largest.

Conceptually:

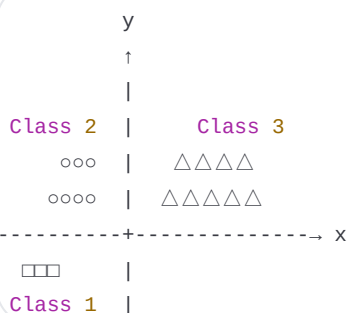
```
scores = [  
    gaussian_discriminant(x, mu1_hat, Sigma1_hat),  
    gaussian_discriminant(x, mu2_hat, Sigma2_hat),  
    gaussian_discriminant(x, mu3_hat, Sigma3_hat)  
]  
  
prediction = np.argmax(scores)
```

Repeat this for every test point.

18. Part 1(f): Decision boundary plot

This part is visually similar to Lab 4.

Imagine your feature space is:



We evaluate the classifier over a grid of (x, y) points.

For every grid point:

Which class would the classifier predict?

Then draw contour lines where the prediction changes.

Those contours become the **decision boundaries**.

Then overlay the test points.

This is exactly what the assignment asks for.

AI5707_Assignment_5

19. Part 1(g): Confusion matrix

Suppose the actual class is:

1 2 3

and predictions are made.

You might get:

	$Pred_1$	$Pred_2$	$Pred_3$
$Actual_1$	1980	20	0
$Actual_2$	15	1960	25
$Actual_3$	0	30	1970

This means:

- Class 1 $\rightarrow$ 1980 correctly classified
- 20 were incorrectly called Class 2
- Class 2 $\rightarrow$ 1960 correctly classified
- etc.

The diagonal entries are the correct predictions.

So:

large diagonal = good classifier

The assignment explicitly requires this table.

AI5707_Assignment_5

20. Part 2 — Real dataset

This is actually the more interesting part.

The real data has:

`x, y, class`

For example:

```
2.14, 1.82, 0
2.34, 1.91, 0
6.21, 5.77, 1
...
```

You load it:

```
data = np.loadtxt(...)
```

or with pandas:

```
df = pd.read_csv(...)
```

Then:

```
x = df[["x", "y"]].values  
y = df["class"].values
```

21. Split real data

Again:

80% $\rightarrow$ *training*

20% $\rightarrow$ *testing*

For example:

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,  
    stratify=y  
)
```

The important idea is **stratification**.

It helps preserve approximately the same class proportions in train and test.

22. Estimate one Gaussian per class

Suppose the real dataset has 3 classes.

Then:

```
Class 0  
     $\mu_0$   
     $\Sigma_0$   
  
Class 1  
     $\mu_1$   
     $\Sigma_1$   
  
Class 2  
     $\mu_2$   
     $\Sigma_2$ 
```

We calculate these from:

```
x_train[y_train == 0]
x_train[y_train == 1]
x_train[y_train == 2]
```

This is a crucial pattern to understand.

23. Then classify

For every point in `x_test` :

```
Compute likelihood under Class 0
Compute likelihood under Class 1
Compute likelihood under Class 2
                        ↓
                    choose maximum
```

Then compare:

`y_test`

against:

`y_pred`

24. Confusion matrix for real data

You can produce:

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
```

For example:

$$\begin{bmatrix} 45 & 2 & 3 \\ 1 & 48 & 1 \\ 4 & 0 & 46 \end{bmatrix}$$

Again, diagonal = correct classification.

25. What your professor probably wants you to observe

Your observations shouldn't just say:

"Accuracy was high."

You want to connect the **geometry** to the **statistics**.

For example:

Same covariance

The estimated Gaussian classes have similar spread, so the discriminant quadratic terms cancel or nearly cancel.

Therefore:

decision boundary is approximately linear

Different covariance

The classes have different spreads/orientations.

Therefore the quadratic terms remain:

decision boundary becomes curved

More samples

Since you're using around 10,000 samples per class, the estimated:

$$\hat{\mu}, \hat{\Sigma}$$

should generally be close to the true parameters used to generate the synthetic data.

That's a very important MLE observation.

26. The entire Assignment 5 in one picture

Think of the whole assignment like this:



And underneath everything is one central idea:

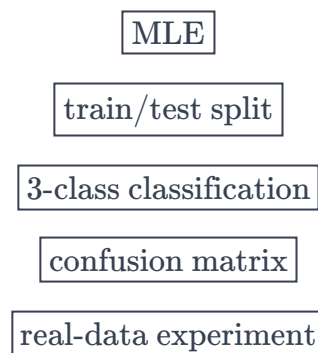
Data → Estimate Gaussian parameters → Classify → Evaluate

27. How this maps directly onto your Lab 4 notebook

Your Lab 4 already gives you:

Lab 4 component	Assignment 5 use
<code>generate_gaussian()</code>	Generate synthetic training/test data
covariance calculations	Compare true vs MLE covariance
eigenvalues/eigenvectors	Analyze estimated covariance
Mahalanobis distance	Gaussian likelihood/discriminant
Gaussian discriminant	Classification
decision-boundary plotting	Assignment boundary requirement
multiple covariance experiments	Same/different covariance comparison

The **new pieces** are mainly:



28. One thing I want you to be very clear about

There are **two different covariance concepts** floating around:

Covariance used to generate the synthetic data

You know it beforehand:

$$\Sigma$$

Covariance estimated from training data

You calculate:

$$\hat{\Sigma}$$

They won't be exactly identical.

For example, theoretically:

$$\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$$

but after finite data you may get:

$$\hat{\Sigma} = \begin{bmatrix} 4.02 & 0.03 \\ 0.03 & 0.98 \end{bmatrix}$$

That's **normal**.

As the number of samples increases, the estimate should generally get closer to the true covariance.

That's one of the main things this assignment is demonstrating.

29. What we should do with your existing code

Rather than writing a completely unrelated notebook, the clean approach is:

```
YOUR LAB 4 CODE
↓
Keep Gaussian generation
↓
Add 3-class data generation
↓
Add train/test split
↓
Add MLE  $\mu$ 
↓
Add MLE  $\Sigma$  using  $1/N$ 
↓
Add 3-class discriminant classifier
↓
Add confusion matrix
↓
Adapt boundary plot
↓
Load real dataset
↓
Repeat everything
```

That will make Assignment 5 look like a natural continuation of your Lab 4 rather than a random new implementation.

One source-specific caveat: your Assignment 5 PDF refers to the **last-assignment generator** and to a real-data file whose suffix matches your team number, but the PDF itself does not contain the  actual real dataset.

AI5707_Assignment_5

help me prepare for VIVA of this experiment

Next, the most useful step is to go through the **actual Assignment 5 code cell-by-cell**, using your uploaded `main(5).ipynb` as the base, and explain exactly what every line is doing and what needs to be changed from Lab 4. Absolutely. For this viva, don't try to memorize the entire notebook. You need to be able to explain the **story of the experiment** and defend the important mathematical choices.

Based on your Assignment 5, the core is:

Generate/obtain data → split into train/test → estimate Gaussian parameters using MLE → build Gaussian classifier → classify test data → plot decision boundaries → evaluate using confusion matrix.

AI5707_Assignment_5

VIVA PREP — Assignment 5

1. The 30-second answer

If the examiner asks:

"What did you do in this experiment?"

Say:

"The objective of this experiment is to perform maximum likelihood estimation of Gaussian parameters and use them for classification. For synthetic data, we generate three Gaussian classes with different covariance structures such as isotropic, diagonal and full covariance. We split the data into 80% training and 20% testing data. From the training data, we estimate the mean and covariance of each class using MLE. Then we use the Gaussian discriminant function to classify the test samples. Finally, we visualize the decision boundaries and evaluate the classifier using a confusion matrix. We repeat the same procedure on the given real 2-D dataset."

AI5707_Assignment_5

Memorize this.

2. Most important viva questions

Q1. What is Maximum Likelihood Estimation?

Answer

Maximum Likelihood Estimation, or MLE, is a method for estimating unknown model parameters by choosing the parameters that maximize the likelihood of observing the given data.

For a Gaussian distribution, we estimate:

$$\mu$$

and

$$\Sigma$$

from the training samples.

Q2. What is the MLE estimate of the mean?

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N x_i$$

In simple words:

The MLE estimate of the Gaussian mean is simply the sample mean.

Python:

```
mu_hat = np.mean(X, axis=0)
```

Q3. What is the MLE estimate of covariance?

This is a **VERY** likely viva question.

$$\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

Notice:

$$N$$

not $N - 1$.

Examiner might ask:

"Why N and not N-1?"

Say:

Because this is the maximum likelihood estimator of covariance. The unbiased sample covariance estimator uses $N - 1$, whereas the Gaussian MLE uses N .

🔥 **Remember this distinction.**

Q4. Why do we estimate parameters using only training data?

Say:

To prevent data leakage. The test data must remain unseen during parameter estimation so that it can provide an unbiased evaluation of the classifier's performance.

In other words:

TRAINING DATA

↓

Estimate μ and Σ

↓

Build model

↓

TEST DATA

↓

Evaluate

Never:

All data → estimate parameters → **test**

because then the test data has influenced the model.

Q5. Why do we use an 80:20 split?

The assignment specifically asks for:

80% → *training*

20% → *testing*

AI5707_Assignment_5

Say:

The larger training portion gives the model enough data to estimate the Gaussian parameters reliably, while the remaining 20% provides unseen data for evaluation.

Q6. Why are there three classes?

Because the experiment studies **multi-class Gaussian classification**.

We have:

C_1, C_2, C_3

For each class we estimate:

$(\hat{\mu}_1, \hat{\Sigma}_1)$

$(\hat{\mu}_2, \hat{\Sigma}_2)$

$(\hat{\mu}_3, \hat{\Sigma}_3)$

Then a test point is assigned to whichever class gives the highest discriminant score.

Q7. What is an isotropic covariance matrix?

This is another likely one.

An isotropic covariance matrix has the form:

$$\Sigma = \sigma^2 I$$

For 2-D:

$$\Sigma = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{bmatrix}$$

Meaning:

- same variance in both dimensions
- zero covariance
- no preferred direction

Therefore the Gaussian contours are **circular**.

One-line answer:

Isotropic means the variance is the same in every direction.

Q8. What is diagonal covariance?

$$\Sigma = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix}$$

Here:

$$\sigma_x^2 \neq \sigma_y^2$$

may be possible.

Therefore the contours are generally **elliptical but axis-aligned**.

Q9. What is full covariance?

Example:

$$\Sigma = \begin{bmatrix} 4 & 1.5 \\ 1.5 & 2 \end{bmatrix}$$

The off-diagonal values are non-zero.

Therefore:

$$\text{Cov}(X, Y) \neq 0$$

which means the two features are correlated.

The Gaussian ellipse can therefore be **rotated**.

Q10. What's the difference between diagonal and full covariance?

	Diagonal	Full
Off-diagonal terms	0	Can be non-zero
Features correlated?	No	Yes
Ellipse rotated?	Generally no	Can be
Matrix	$\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}$	$\begin{bmatrix} a & c \\ c & b \end{bmatrix}$

Viva answer:

Diagonal covariance assumes zero covariance between features, whereas full covariance allows correlations between features.

3. Gaussian classifier questions

Q11. What Gaussian probability density did you use?

For d -dimensional data:

$$p(x|C_k) = \frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}} e^{-\frac{1}{2}(x-\mu_k)^T \Sigma_k^{-1} (x-\mu_k)}$$

For our experiment:

$$d = 2$$

because the data is 2-D.

Q12. What does this term mean?

$$(x - \mu)^T \Sigma^{-1} (x - \mu)$$

This is the **squared Mahalanobis distance**.

$$D_M^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$$

Why is it useful?

Unlike Euclidean distance, Mahalanobis distance considers:

- variance
- covariance
- orientation of the distribution.

So it is particularly suitable for Gaussian distributions.

Q13. Why not just use Euclidean distance?

Excellent viva question.

Say:

Euclidean distance treats all directions equally. Mahalanobis distance accounts for the covariance structure of the data, so it measures distance relative to the spread and correlation of the Gaussian distribution.

Q14. What is the discriminant function?

A commonly used Gaussian discriminant is:

$$g_k(x) = -\frac{1}{2} \ln |\Sigma_k| - \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) + \ln P(C_k)$$

For equal priors:

$$P(C_1) = P(C_2) = P(C_3)$$

the prior term is the same for all classes and doesn't affect the winner.

Therefore we select:

$$\arg \max_k g_k(x)$$

Q15. Why use log probability?

This is a classic question.

Say:

Gaussian probabilities can become extremely small, especially in higher dimensions. Taking logarithms prevents numerical underflow and also converts products into sums, making computation more stable.

Also:

$$\arg \max p(x|C_k) = \arg \max \log p(x|C_k)$$

because logarithm is a monotonically increasing function.

4. Decision boundary questions 🔥

This section is especially important because your previous Lab 4 dealt with decision boundaries.

Q16. What is a decision boundary?

It is the set of points in feature space where two or more classes have equal discriminant scores, so the classifier is indifferent between those classes.

For two classes:

$$g_1(x) = g_2(x)$$

defines the boundary.

Q17. Why is the boundary linear when covariance is the same?

Suppose:

$$\Sigma_1 = \Sigma_2 = \Sigma$$

When we compare the two Gaussian discriminants, the quadratic terms involving:

$$x^T \Sigma^{-1} x$$

cancel.

What's left is a linear function of x .

Therefore:

linear decision boundary

This is the fundamental idea behind **LDA**.

Q18. Why is the boundary quadratic when covariance differs?

If:

$$\Sigma_1 \neq \Sigma_2$$

the quadratic terms don't cancel.

Therefore the boundary contains terms such as:

$$x^T A x$$

and becomes:

quadratic / curved

This corresponds to **QDA**.

Q19. LDA vs QDA?

🔥 Very likely.

LDA	QDA
Same covariance across classes	Different covariance for each class
Linear boundary	Quadratic boundary
Fewer parameters	More parameters
Simpler model	More flexible model

Perfect viva answer:

LDA assumes a common covariance matrix for all classes, resulting in linear decision boundaries. QDA estimates a separate covariance matrix for each class, allowing quadratic decision boundaries.

5. Synthetic-data questions

Q20. Why generate synthetic Gaussian data?

Say:

Synthetic data allows us to know the true parameters beforehand. Therefore, after estimating the parameters using MLE, we can compare the estimated mean and covariance with the actual values used to generate the data.

That's an important experimental advantage.

Q21. What happens as the number of samples increases?

Generally:

$$N \uparrow$$

causes the estimates:

$$\hat{\mu}, \hat{\Sigma}$$

to become more stable and closer to the true parameters.

This is related to the consistency of MLE.

Viva answer:

With more samples, the estimated parameters generally converge toward the true Gaussian parameters.

Q22. Why does estimated covariance not exactly equal the true covariance?

Because you're estimating from a **finite sample**.

For example:

True:

$$\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 2 \end{bmatrix}$$

Estimated:

$$\hat{\Sigma} = \begin{bmatrix} 4.01 & 0.02 \\ 0.02 & 1.98 \end{bmatrix}$$

The difference is due to sampling variation.

6. Confusion matrix questions

Q23. What is a confusion matrix?

It compares:

Actual labels

against:

Predicted labels

For three classes:

	P_1	P_2	P_3
A_1			
A_2			
A_3			

The diagonal represents correctly classified samples.

Q24. What does a large diagonal mean?

High values along the diagonal indicate that most samples are classified correctly.

Q25. What does an off-diagonal value mean?

It indicates a misclassification.

For example:

$$CM_{1,2} = 20$$

means:

20 samples that actually belong to Class 1 were predicted as Class 2.

Q26. How do you calculate accuracy?

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total predictions}}$$

From the confusion matrix:

$$Accuracy = \frac{\sum_i CM_{ii}}{\sum_{i,j} CM_{ij}}$$

7. Real dataset questions

Q27. Why did you use the real dataset?

To check whether the Gaussian MLE-based classifier that worked on controlled synthetic data also works on actual data.

The assignment explicitly requires repeating the procedure on the supplied real 2-D dataset.

A15707_Assignment_5

Q28. Why might accuracy on real data be lower?

Good answer:

Real data may not follow a perfect Gaussian distribution. There may also be noise, overlapping classes, outliers, or correlations that aren't well represented by the Gaussian model.

Q29. What assumptions does the Gaussian classifier make?

Important.

It assumes that each class can reasonably be modeled by a multivariate Gaussian distribution:

$$X|C_k \sim N(\mu_k, \Sigma_k)$$

If the real data is very non-Gaussian, the model may perform poorly.

8. Coding viva questions

Your examiner may point at code and ask:

"What does this do?"

```
mu = np.mean(X, axis=0)
```

Answer:

It calculates the mean vector across the samples. `axis=0` means the mean is calculated separately for each feature.

"What does this do?"

```
diff = X - mu
```

Answer:

It centers the data by subtracting the estimated mean from every sample.

"What does this do?"

```
Sigma = diff.T @ diff / len(X)
```

Answer:

It calculates the MLE covariance matrix. `diff.T @ diff` accumulates the outer products of the centered samples, and division by N gives the MLE covariance.

"Why transpose?"

Because:

$$X_c^T X_c$$

must produce a:

$$2 \times 2$$

covariance matrix for 2-D features.

9. A sneaky question: Why must covariance be positive definite?

For a valid multivariate Gaussian covariance matrix, it should be positive semidefinite; for the usual nonsingular Gaussian density, we need it to be positive definite.

Why?

Because the Gaussian formula requires:

$$|\Sigma|$$

and

$$\Sigma^{-1}$$

A singular covariance matrix cannot be inverted normally.

10. Another sneaky question: Why do we need the determinant?

In:

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2} D_M^2}$$

the determinant:

$$|\Sigma|$$

represents the overall volume/spread of the Gaussian distribution.

It is also important in the discriminant because two classes may have different covariance spreads.

11. Why can't we only use Mahalanobis distance?

This is a **nice tricky question**.

If the covariance matrices are different, the Gaussian discriminant contains:

$$-\frac{1}{2} \ln |\Sigma_k|$$

in addition to the Mahalanobis term.

So classification isn't simply:

choose the smallest Mahalanobis distance.

The covariance determinant also affects the likelihood.

12. Your "examiner rapid-fire" round ⚡

Try answering these **without looking at the answers**.

Q1

What is MLE?

Q2

What is the MLE of Gaussian mean?

Q3

What is the denominator in MLE covariance?

Q4

Why N , not $N - 1$?

Q5

What is isotropic covariance?

Q6

What does an off-diagonal covariance term represent?

Q7

What is Mahalanobis distance?

Q8

Why is Mahalanobis better than Euclidean here?

Q9

What happens when all classes have the same covariance?

Q10

What happens when covariance differs between classes?

Q11

LDA vs QDA?

Q12

Why split into train and test?

Q13

Why estimate parameters only from training data?

Q14

What is the purpose of the confusion matrix?

Q15

What does a diagonal element represent?

Q16

Why use log likelihood?

Q17

What happens when sample size increases?

Q18

Why might real data perform worse than synthetic data?

Q19

What does $|\Sigma|$ mean?

Q20

What happens if Σ is singular?



The 10 things I would memorize tonight

If your viva is tomorrow and you don't have time for everything, memorize these:

1. MLE mean

$$\hat{\mu} = \frac{1}{N} \sum x_i$$

2. MLE covariance

$$\hat{\Sigma} = \frac{1}{N} \sum (x_i - \hat{\mu})(x_i - \hat{\mu})^T$$

3. MLE uses N , unbiased covariance uses $N - 1$.

4. Mahalanobis distance

$$D^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$$

5. Isotropic

$$\Sigma = \sigma^2 I$$

6. Same covariance $\rightarrow$ linear boundary $\rightarrow$ LDA

7. **Different covariance → quadratic boundary → QDA**
8. **80% training / 20% testing**
9. **Estimate parameters only using training data.**
10. **Confusion-matrix diagonal = correct classifications.**

If you can explain those ten naturally, you're already in pretty good shape.

🔥 One last thing: the viva "why?"

Your examiner is very likely to keep asking "why?" after your first answer.

For example:

Examiner: Why MLE?

You: To estimate the Gaussian parameters from the observed training data.

Examiner: Why only training data?

You: To avoid data leakage and evaluate on unseen samples.

Examiner: Why Gaussian?

You: The experiment assumes that each class can be modeled as a Gaussian distribution.

Examiner: Why covariance?

You: Because it captures the spread and correlation structure of the features.

Examiner: Why different covariance?

You: To study how class-specific covariance affects the discriminant and produces quadratic boundaries.

Examiner: Why confusion matrix?

You: To see not only the overall accuracy but also which classes are being confused.

That's the style you want: **short → precise → mathematical when needed**. Don't dump a textbook on the examiner. 

 even more depth questions, testing the fundamentals of this experiment



Yes — let's go **much deeper**. For a PRML viva, the examiner can easily move beyond "what is MLE?" and test whether you actually understand **why every equation and every coding step exists**.

I'll structure this like an aggressive viva: **question → what they're testing → strong answer → possible follow-up**.

🔥 Deep Viva: MLE + Gaussian Classification

PART A — FUNDAMENTALS OF PROBABILITY

1. What exactly are you estimating in this experiment?

What they're testing: Do you understand what "parameter estimation" means?

Answer:

For each class C_k , we're assuming:

$$X|C_k \sim \mathcal{N}(\mu_k, \Sigma_k)$$

The unknown parameters are:

$$\mu_k, \Sigma_k$$

We estimate them from the training samples:

$$\hat{\mu}_k, \hat{\Sigma}_k$$

So we're not estimating the class labels. We're estimating the **parameters of the probability distribution associated with each class**.

2. What is the difference between a random variable and an observation?

A random variable X represents the underlying stochastic quantity.

An observation x_i is one realized value of X .

For example:

$$X \sim N(\mu, \sigma^2)$$

and after sampling we might observe:

$$x_1 = 2.1, \quad x_2 = 1.8, \quad x_3 = 2.7$$

The x_i 's are observations used to estimate the unknown parameters.

PART B — LIKELIHOOD VS PROBABILITY

3. What's the difference between probability and likelihood?

🔥 **Very good viva question.**

Suppose:

$$p(x|\theta)$$

Probability viewpoint:

Parameters θ are fixed and we ask:

How probable is observing x ?

Likelihood viewpoint:

Observed x is fixed and we ask:

Which value of θ makes this observation most likely?

So:

Probability: vary data

Likelihood: vary parameters

The mathematical expression can be the same, but the interpretation is different.

4. Why do we maximize likelihood instead of probability?

Because in the experiment the **data is already observed**.

We don't control x .

We want to determine:

$$\mu, \Sigma$$

that best explain the observed training data.

Therefore:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} L(\theta|X)$$

PART C — DERIVING MLE

5. Can you derive the MLE of the Gaussian mean?

This is the kind of question that separates "I memorized the formula" from "I understand it."

Suppose:

$$x_i \sim N(\mu, \sigma^2)$$

The likelihood is:

$$L(\mu, \sigma^2) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

Take log:

$$\ell = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2$$

Differentiate with respect to μ :

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^N (x_i - \mu)$$

Set to zero:

$$\sum (x_i - \mu) = 0$$

Therefore:

$$\hat{\mu} = \frac{1}{N} \sum x_i$$

So the sample mean is the Gaussian MLE.

6. Why do we take logarithm of the likelihood?

Three important reasons:

1. Product becomes sum

$$\log \prod_i p(x_i) = \sum_i \log p(x_i)$$

2. Easier differentiation

Sums are easier to differentiate than products.

3. Numerical stability

Multiplying many small probabilities can cause numerical underflow.

Therefore we maximize:

$$\log L$$

instead of L .

7. Does taking logarithm change the MLE?

No.

Because logarithm is strictly increasing:

$$a > b \iff \log a > \log b$$

Therefore:

$$\arg \max_{\theta} L(\theta) = \arg \max_{\theta} \log L(\theta)$$

PART D — COVARIANCE

8. Why is covariance a matrix?

Because your feature vector is multidimensional.

For:

$$X = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$$

we need:

$$\text{Cov}(X) = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) \end{bmatrix}$$

So covariance tells us:

- variance of each feature
- covariance between every pair of features.

9. What does a positive covariance mean?

If:

$$\text{Cov}(X, Y) > 0$$

then, generally, when X increases, Y tends to increase.

Negative covariance means they tend to move in opposite directions.

Zero covariance means there is no **linear covariance**.

⚠ Don't say:

"Zero covariance means the variables are independent."

That's not generally true.

For Gaussian variables, however, zero covariance implies independence under the appropriate jointly Gaussian assumption.

10. Why is covariance symmetric?

Because:

$$\text{Cov}(X_i, X_j) = \text{Cov}(X_j, X_i)$$

Therefore:

$$\boxed{\Sigma = \Sigma^T}$$

This is why covariance matrices are symmetric.

11. Why is covariance positive semidefinite?

For any vector a :

$$a^T \Sigma a = \text{Var}(a^T X)$$

Variance cannot be negative.

Therefore:

$$a^T \Sigma a \geq 0$$

So covariance matrices are positive semidefinite.

12. Why do we need Σ^{-1} ?

Look at Mahalanobis distance:

$$D_M^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$$

The inverse covariance effectively **normalizes distances according to the spread and correlation of the data**.

If a direction has huge variance, deviations in that direction are penalized less.

If a direction has small variance, deviations are penalized more.

PART E — GEOMETRY

13. Why are Gaussian contours ellipses?

A constant-density Gaussian contour satisfies:

$$(x - \mu)^T \Sigma^{-1} (x - \mu) = c$$

This is a quadratic equation.

For positive-definite Σ , the resulting shape is an ellipse in 2-D.

14. What determines the orientation of the ellipse?

The eigenvectors of Σ .

If:

$$\Sigma v_i = \lambda_i v_i$$

then:

- v_i gives the principal direction

- λ_i gives the variance along that direction.

So:

Eigenvectors $\rightarrow$ orientation

Eigenvalues $\rightarrow$ spread

This directly connects your Assignment 5 to the covariance/eigenvalue analysis from your previous experiment.

15. What happens if both eigenvalues are equal?

Suppose:

$$\lambda_1 = \lambda_2$$

Then the distribution has equal variance in every direction.

Geometrically, the contour becomes a circle.

This is the geometric interpretation of isotropic covariance:

$$\Sigma = \sigma^2 I$$

16. Why does full covariance rotate the ellipse?

Because non-zero off-diagonal terms indicate correlation.

The eigenvectors are no longer necessarily aligned with the x - and y -axes.

Hence the principal axes rotate.

PART F — GAUSSIAN CLASSIFIER

17. Where does the classifier actually come from?

Start with Bayes' theorem:

$$P(C_k|x) = \frac{p(x|C_k)P(C_k)}{p(x)}$$

For classification, $p(x)$ is common to every class.

Therefore:

$$P(C_k|x) \propto p(x|C_k)P(C_k)$$

So we choose:

$$\arg \max_k p(x|C_k)P(C_k)$$

That is the foundation of the classifier.

18. What's the role of the prior probability?

$$P(C_k)$$

represents how likely class k is **before seeing the sample**.

If all classes have equal prior probabilities:

$$P(C_1) = P(C_2) = P(C_3)$$

then the priors don't affect which class wins.

But if classes have unequal priors, they influence the classification.

19. What if Class 1 has prior probability 0.9 and Class 2 has 0.1?

Even if:

$$p(x|C_1) = p(x|C_2)$$

we would prefer Class 1 because:

$$P(C_1|x) \propto 0.9$$

while:

$$P(C_2|x) \propto 0.1$$

So priors can shift the decision boundary.

20. Why does equal covariance produce a linear boundary?

Let's derive the key idea.

The Gaussian discriminant contains:

$$-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)$$

If:

$$\Sigma_1 = \Sigma_2 = \Sigma$$

then both classes contain the same quadratic term:

$$-\frac{1}{2}x^T \Sigma^{-1} x$$

When calculating:

$$g_1(x) - g_2(x)$$

that term cancels.

We're left with terms linear in x .

Hence:

linear boundary

21. Why does different covariance produce a quadratic boundary?

If:

$$\Sigma_1 \neq \Sigma_2$$

then:

$$x^T \Sigma_1^{-1} x$$

and

$$x^T \Sigma_2^{-1} x$$

are different.

They don't cancel.

Therefore the boundary contains quadratic terms.

quadratic boundary

PART G — LDA / QDA

22. Is your experiment actually doing LDA or QDA?

Answer carefully:

It depends on the covariance assumption.

If we assume a common covariance matrix:

$$\Sigma_1 = \Sigma_2 = \Sigma_3$$

the resulting Gaussian discriminant corresponds to the LDA setting.

If every class gets its own covariance:

$$\Sigma_1, \Sigma_2, \Sigma_3$$

it corresponds to the QDA setting.

23. Why does QDA require more data?

Because QDA estimates a separate covariance matrix for every class.

For a d -dimensional Gaussian:

Mean parameters:

$$d$$

Covariance parameters:

Because covariance is symmetric:

$$\frac{d(d+1)}{2}$$

Therefore each class requires:

$$d + \frac{d(d+1)}{2}$$

parameters.

For 2-D:

$$2 + \frac{2(3)}{2} = 5$$

So each Gaussian class has five independent parameters:

$$\mu_x, \mu_y, \sigma_x^2, \sigma_y^2, \sigma_{xy}$$

QDA estimates these separately for each class.

24. What happens if you have too few training samples?

Your covariance estimate can become unstable.

In extreme cases, covariance can become singular:

$$|\Sigma| = 0$$

and then:

$$\Sigma^{-1}$$

doesn't exist.

This can cause the Gaussian classifier to fail numerically.

25. Why might we use regularization?

One common approach is:

$$\Sigma_{\text{regularized}} = \Sigma + \lambda I$$

where:

$$\lambda > 0$$

This improves numerical stability and can prevent near-singular covariance matrices.

If asked whether your assignment necessarily requires this, don't claim it does unless your code/assignment specifies it. It's a general improvement.

PART H — TRAIN/TEST FUNDAMENTALS

26. Why not use all 30,000 synthetic samples for training?

Because then we wouldn't have an independent test set.

The goal isn't merely to fit the generated data.

It's to determine whether the estimated model can correctly classify **unseen samples**.

27. Why should we not estimate μ and Σ using test data?

Because then the test data influences the model.

That's data leakage.

The correct pipeline is:

$$Training \rightarrow Estimate \rightarrow Model \rightarrow Test \rightarrow Evaluate$$

28. Is 80/20 theoretically mandatory?

No.

80/20 is a practical experimental choice specified by this assignment.

Other problems might use:

- 70/30
- 75/25
- 90/10
- cross-validation.

The fundamental requirement is that evaluation data should be independent from parameter estimation.

PART I — SYNTHETIC DATA

29. Why is synthetic data useful for understanding MLE?

Because we know the true parameters.

Suppose we generate data using:

$$\mu = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

and:

$$\Sigma = \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix}$$

After MLE, we might obtain:

$$\hat{\mu} = \begin{bmatrix} 2.01 \\ 2.98 \end{bmatrix}$$

and:

$$\hat{\Sigma} \approx \begin{bmatrix} 3.97 & 1.02 \\ 1.02 & 2.01 \end{bmatrix}$$

We can directly compare:

$$(\mu, \Sigma) \quad \text{vs} \quad (\hat{\mu}, \hat{\Sigma})$$

With real data, we don't know the true generating Gaussian parameters.

30. What happens if the number of samples goes to infinity?

Under standard regularity assumptions:

$$\boxed{\hat{\theta}_{MLE} \rightarrow \theta}$$

This is called **consistency**.

So as:

$$N \rightarrow \infty$$

the estimated parameters approach the true parameters.

PART J — REAL DATA

31. Why might the Gaussian model fail on real data?

Because the assumption:

$$X|C_k \sim N(\mu_k, \Sigma_k)$$

may be incorrect.

Real classes could have:

- skewed distributions
- multimodal distributions
- outliers
- nonlinear structures
- heavy tails
- class overlap.

Then a single Gaussian per class may not describe the data adequately.

32. If the real data isn't Gaussian, does MLE stop working?

No.

This is a subtle point.

MLE can still estimate the parameters of the **chosen model**, but if the model is wrong, those parameters may not represent the real distribution well.

So there are two separate questions:

1. **Can we mathematically estimate the parameters?**
2. **Is the assumed Gaussian model appropriate?**

The first can be yes while the second is no.

PART K — CONFUSION MATRIX

33. Why isn't accuracy enough?

Suppose:

Class 1 → 95%
Class 2 → 60%
Class 3 → 92%

An overall accuracy might hide the fact that Class 2 is performing badly.

The confusion matrix shows **which classes are being confused with which**.

34. What does a confusion matrix tell you that accuracy doesn't?

It tells you the **direction of errors**.

For example:

$$CM_{1,2} = 50$$

means Class 1 is often mistaken for Class 2.

That's much more informative than simply saying:

Accuracy = 92%.

PART L — NUMERICAL/CODING FUNDAMENTALS

35. Why do we use matrix multiplication here?

The Mahalanobis term is:

$$(x - \mu)^T \Sigma^{-1} (x - \mu)$$

This is a quadratic form.

It produces a scalar:

$$1 \times d \cdot d \times d \cdot d \times 1 = 1 \times 1$$

For 2-D:

$$(1 \times 2)(2 \times 2)(2 \times 1)$$

gives:

$$1 \times 1$$

36. Why can't you use ordinary element-wise multiplication?

Because the expression represents a quadratic form.

We need:

$$v^T A v$$

not:

$$v * A * v$$

in the element-wise sense.

That's why NumPy uses matrix multiplication:

```
diff.T @ inv_cov @ diff
```

37. Why should we avoid explicitly computing the inverse?

A strong numerical-computing answer:

Although the mathematical formula contains Σ^{-1} , explicitly computing a matrix inverse is generally less numerically stable and less efficient than solving a linear system.

Instead of:

```
inv_cov = np.linalg.inv(Sigma)
```

one can use a solver such as:

```
np.linalg.solve(Sigma, diff)
```

when implementing a robust version.

For a classroom implementation, explicit inverse may still be perfectly acceptable if that's how the experiment is specified.

38. Why must Σ be invertible?

Because the Gaussian density contains:

$$\Sigma^{-1}$$

and:

$$|\Sigma|^{-1/2}$$

If:

$$|\Sigma| = 0$$

then Σ is singular.

The ordinary multivariate Gaussian density formula breaks down.

PART M — REALLY TRICKY QUESTIONS

39. Is MLE always unbiased?

No.

This is an excellent question.

For Gaussian covariance:

$$\hat{\Sigma}_{MLE} = \frac{1}{N} \sum (x_i - \bar{x})(x_i - \bar{x})^T$$

is generally **biased**.

The $N - 1$ estimator is unbiased for covariance under the usual assumptions.

So:

MLE does not necessarily mean unbiased

40. Then why use MLE if it is biased?

Because MLE has desirable properties such as:

- consistency
- asymptotic efficiency under regular conditions
- natural likelihood-based interpretation.

For large datasets, the bias of the Gaussian covariance MLE becomes relatively small.

And importantly, **this experiment specifically asks for MLE.**

41. Does MLE always give the true parameters?

No.

For finite data:

$$\hat{\theta} \neq \theta$$

in general.

MLE provides an estimate.

Under suitable conditions:

$$\hat{\theta} \rightarrow \theta$$

as sample size increases.

42. Does maximum likelihood mean maximum probability of the parameter?

No.

This is another trap.

Parameters aren't being assigned probabilities in ordinary MLE.

We're finding the parameter value that maximizes the likelihood of the observed data:

$$\hat{\theta} = \arg \max_{\theta} p(X|\theta)$$

43. Is the likelihood itself a probability distribution over parameters?

No.

Likelihood is a function of the parameters for fixed data.

It doesn't have to integrate to 1 over the parameter space.

44. Why does the Gaussian classifier choose the maximum discriminant?

Because:

$$g_k(x)$$

is proportional to the log posterior, up to terms common to all classes.

Thus:

$$\hat{C} = \arg \max_k g_k(x)$$

gives the **maximum a posteriori (MAP)** class under the model.

With equal priors, MAP reduces to maximum likelihood classification.

45. What's the difference between ML classification and MAP classification?

Maximum likelihood classification:

$$\arg \max_k p(x|C_k)$$

MAP classification:

$$\arg \max_k p(x|C_k)P(C_k)$$

MAP considers class priors.

If priors are equal:

$$ML = MAP$$

in terms of the selected class.

Ultimate viva challenge

If your examiner says:

"Explain your entire experiment mathematically without referring to your code."

Give this answer:

"We assume that the feature vector conditioned on each class follows a multivariate Gaussian distribution:

$$X|C_k \sim N(\mu_k, \Sigma_k).$$

We divide the available data into training and testing sets. For each class, using only the training samples, we estimate the Gaussian mean using

$$\hat{\mu}_k = \frac{1}{N} \sum_i x_i$$

and the covariance using the MLE

$$\hat{\Sigma}_k = \frac{1}{N} \sum_i (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T.$$

We then use the Gaussian likelihood together with the class prior to obtain a discriminant function. For every test point, we calculate the discriminant for each class and assign the point to the class with the maximum score.

When all classes share the same covariance, the quadratic terms in the discriminants cancel, producing linear decision boundaries. When each class has its own covariance, the quadratic terms remain and the boundaries can become quadratic.

Finally, we compare predicted and actual test labels using a confusion matrix and analyze the classification performance. We repeat this process for the synthetic Gaussian data and the given real 2-D dataset."

That answer demonstrates that you understand the **entire chain**, rather than memorizing isolated formulas.

The examiner's likely "trap" questions

Before your viva, make sure you can answer these **without hesitation**:

Question	Key thing to say
MLE covariance denominator?	N
Unbiased covariance denominator?	$N - 1$
MLE always unbiased?	No
Probability vs likelihood?	Data vs parameter viewpoint
Why log likelihood?	Easier + numerical stability
Isotropic covariance?	$\sigma^2 I$
Off-diagonal covariance?	Feature correlation
Covariance eigenvectors?	Principal directions
Covariance eigenvalues?	Variance along those directions
Same covariance?	Linear boundary
Different covariance?	Quadratic boundary

Question	Key thing to say
Same covariance classifier?	LDA setting
Different covariance classifier?	QDA setting
Mahalanobis distance?	Covariance-aware distance
Why train/test?	Unseen-data evaluation
Why training only for MLE?	Avoid leakage
Why synthetic data?	True parameters known
Why real data?	Test model assumptions on actual data
Why confusion matrix?	Understand class-specific errors
Singular covariance?	Inverse/determinant problem
More samples?	Estimates become more stable/consistent
Equal priors?	ML and MAP give same class decision

If you can handle **these + the previous set**, you're no longer just prepared for “what did you do?” questions—you can handle an examiner who starts digging into **probability** → **MLE** → **covariance** → **geometry** → **Bayes** → **discriminants** → **LDA/QDA** → **numerical implementation**.

If you want, I can:

- Explain how QDA and LDA differ in practice with covariance assumptions
- Describe numerical challenges with covariance inversion and solutions
- Discuss why Gaussian assumptions might fail on real data and effects on MLE